Solar-Mixer: An Efficient End-to-End Model for Long-Sequence Photovoltaic Power Generation Time Series Forecasting

Ziyuan Zhang, Jianzhou Wang, Yurui Xia, Danxiang Wei, and Yunbo Niu

***Abstract*—The expansion of photovoltaic power generation makes photovoltaic power forecasting an essential requirement. With the development of deep learning, more accurate predictions have become possible. This paper proposes an efficient end-to-end model for solar power generation that allows for long-sequence time series forecasting. Two modules comprise the forecasting model: the anomaly detection module and the forecasting module. Singular values are detected and corrected by the anomaly detection module. And in the forecasting module, the series is first divided into different intervals. After that, the divided data is passed through a decomposition module embedded in the neural network, and each interval is encoded as high-dimension vectors. These vectors are fed into the mixing layer, which is used to learn the relationship between intervals and channels after the encoding process. Every mixing layer comprises two different layers: the channel-mixing layer allows communication between different channels, and the interval-mixing layer allows communication between different intervals. This neural network is purely dependent on the multi-layer perceptron, resulting in a network with low system latency and low training cost. The experiments results not only show the model proposed in this paper can make accurate long-sequence time series forecasts for photovoltaic power generation at three sites but also show that the model can defeat the state-of-the-art model in long-sequence photovoltaic power generation time series forecasting work.**

***Index Terms*—Photovoltaic generation, long-sequence forecasting, time series forecasting, neural network, lightweight networks.**

I. Introduction

LARGE-SCALE renewable energy production is critical for power systems with the suggestion of carbon emission reduction targets by 2050. However, because renewable energy is uncontrollable and uncertain, bringing large-scale renewable energy such as solar energy or wind energy to the grid will undoubtedly cause the grid system to become unstable [1]. As an example, even with the most stable and reliable renewable energy source, hydro energy, resource shortages in extreme cases still led to a massive power outage in Chengdu, a major city in southwestern China, in the summer of 2022. At the same time, photovoltaic (PV) power generation has potential and is becoming increasingly significant in the power system [2], so accurate PV power forecasting (PPF) is a practical issue for realizing smart grids.

The majority of PPF technologies currently in use target the short and ultra-short term. These technologies have drawn the attention of researchers around the world and have demonstrated satisfactory accuracy. For instance, Yao et al. [3] use advanced U-net and an encoder-decoder architecture to solve the PPF problem. This model has the ability to combine multiple data sources (e.g., time series records and satellite images) and obtain high-performance results. And Simeunović et al. [4] use spatio-temporal graph neural networks for multi-site PPF tasks. This work considers the spatial relationships between PV sites and attempts to model them, which has significant practical value. Although the above work has been able to address short-term PV forecasting tasks well, the reality is that the grid system operation needs a much longer time to deal with the changes in PV generation. Therefore, relative long term PPF is also particularly critical in the construction of smart grids.

In contrast to short-term PPF, long-term PPF generally refers to a forecasting period of several months to several years. The model proposed in this paper performs long-sequence forecasting, which is different from long-term forecasting. For long-sequence forecasting, its definition we refer to ***Informer*** [5]. Specifically, forecasts within 48 steps will be referred to as short-term forecasting, while those above 48 steps will be described as long-sequence forecasting. As an example, if the granularity of time required for a forecast is 0.5 hours, then a forecast of 2 days or more in advance is a long-sequence forecast.

Compared to short-term PFF, long-sequence PPF has its own challenges due to its complexity and inherent trends that are difficult to detect [6]. What makes things more complicated is that weather, solar irradiance level, and other factors affect the performance of PPF. The superposition of multiple influencing factors turns this problem into a multivariate long-sequence time series forecasting task.

Much less work has been done for long-sequence PPF compared to short-term PPF. The existing methods can be divided into the following three categories: statistical-based methods; machine learning (excluding deep learning) based methods; and

Manuscript received 13 October 2022; revised 27 January 2023 and 8 March 2023; accepted 14 April 2023. Date of publication 18 April 2023; date of current version 20 September 2023. This work was supported by the National Natural Science Foundation of China under Grant 71671029. Paper no. TSTE-01056-2022. *(Corresponding author: Jianzhou Wang.)*

The authors are with the Institute of Systems Engineering, Macau University of Science and Technology, Macau 999078, China (e-mail: zhangziyuan960817@gmail.com; jzwang@must.edu.mo; xiayurui99@gmail.com; danxiang130@gmail.com; yunboniu@gmail.com).

Color versions of one or more figures in this article are available at https://doi.org/10.1109/TSTE.2023.3268100.

Digital Object Identifier 10.1109/TSTE.2023.3268100

1949-3029 © 2023 IEEE. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Mukesh Patel School of Technology & Engineering. Downloaded on July 21,2026 at 04:29:54 UTC from IEEE Xplore. Restrictions apply.

deep learning-based methods. For statistical models, the autoregressive moving average (**ARMA**) model [7], the autoregressive integrated moving average (**ARIMA**) model [8], and the seasonal autoregressive integrated moving average (**SARIMA**) [9] are commonly used to deal with this task. The problem with **ARMA**, **ARIMA**, and **SARIMA** models is that the models themselves are for univariate time series, and even though with some improvements they can handle multivariate time series forecasting issues, their forecasting precision is still not reliable enough. Machine learning models such as support vector regression (**SVR**) [10], random forest regression (**RFR**) [11], light gradient boosting machine (**LightGBM**) [12], eXtreme gradient boosting (**XGboost**) [13], etc., which are widely used in data science competitions such as *Kaggle*, can also be used for long-sequence PPF. These models are actually well suited to handling multivariate time series but do not model the potential correlation of historical data or historical intervals with subsequent data, thus leading to a failure to achieve better results in long-sequence time series forecasting.

There has also been a lot of research in recent years attempting long-sequence multivariate time series forecasting using deep learning approaches. Notable methods are mainly long-short term memory (**LSTM**)-based methods [14], [15], convolution-based methods [16], [17], and methods based on the self-attention mechanism. Among them, long-sequence time series forecasting methods based on self-attention mechanism have been proposed a lot, such as **Informer** [5], **Autoformer** [18], **FEDformer** [19], **Pyraformer** [20] etc., and have shown positive results. Admittedly, the self-attention mechanisms are advisable for sequence modeling [21], which is the original intention of its design. These models have achieved extraordinary results in various complex time series forecasting tasks, but we still remain unaware of the appropriateness of these models for long-sequence PPF. On the other hand, self-attention leads to models that require a great deal of training data and also have a massive demand for computational resources. In a real-world scenario, firstly, there is not always such a large amount of data to train such a model, and secondly, there are not always sufficient computational resources to train such a model.

To overcome the drawbacks of the above model, a new end-to-end model is proposed in this paper. First, the raw multivariate PV time series data is fed to the anomaly detection module. This module corrects the values that do not conform to objective patterns (e.g., the wind direction variable, which should have values between 0° and 360°, while some values in the raw data exceed 360°), and then the initially corrected time series are subjected to outlier detection. The detected outliers are further analyzed and then corrected with LightGBM to reduce information loss. Second, the preprocessed data will be entered into the forecasting network. The network first divides the input time series into multiple time intervals. Immediately afterward, the decomposition module decomposes the series, and the decomposed trend terms are joined to the data for further usage. Then, every interval was encoded as a high-dimensional vector. After that, these high-dimensional vectors are fed into the mixer-layer, and after several inter-interval communications and

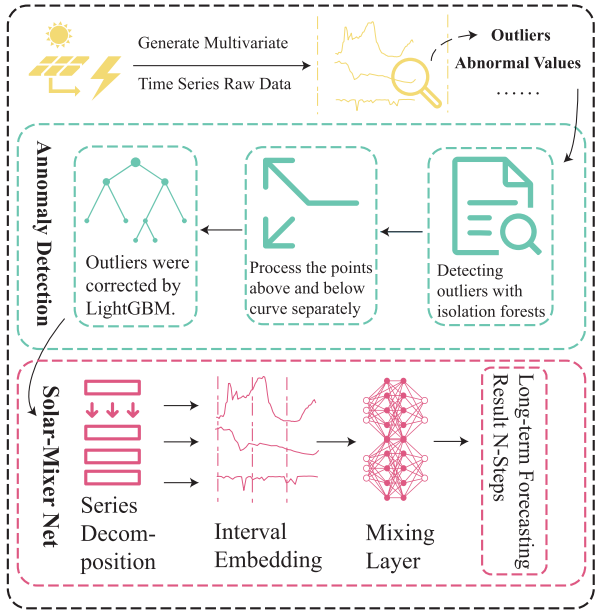


Fig. 1. The overall schematic of the model, and the specific algorithms of each module will be described in detail later.

inter-channel communications, the final n-step forecast results are generated through the fully connected layer. The structure diagram of the whole model is shown in Fig. 1. The model can flexibly handle multivariate time series with a different number of variables, is robust to dirty data, does not use the self-attention mechanism, and can capture the relationship between historical intervals and subsequent data. It has a great advantage in model size and system latency compared to other models while achieving the best results. The main contributions of this paper are summarized as follows:

1) A conservative anomaly detection method that tries to lose less real information was developed and used to perform anomaly detection and correction for PV time series.
2) A novel neural network model was proposed to perform both multivariate and univariate long-sequence PV time series forecasting. The model is easy to train, requires fewer data, and has high performance.
3) Experimental results show that the model proposed in this paper has the best results in several metrics when the model is simpler than the competition when compared to state-of-the-art long-sequence time series forecasting models.

The rest of the paper is organized as follows: For illustration purposes, Section II describes the basic information of the experimental data. Section III describes how the problem was formulated. A detailed forecasting model implementation is introduced in Section IV. Section V contains information about the datasets, hyperparameter settings, evaluation metrics, and experiment details. The results of the ablation experiments will be presented in Section VI. And, the comparison of the model output results with the state-of-the-art model will also be demonstrated in this section. Section VII includes the main conclusions.

Authorized licensed use limited to: Mukesh Patel School of Technology & Engineering. Downloaded on July 21,2026 at 04:29:54 UTC from IEEE Xplore. Restrictions apply.

TABLE I
ABBREVIATION DESCRIPTION OF PV POWER DATA

Abbreviations	Full Name
PR	Performance Ratio
WTC	Weather Temperature Celsius
WRH	Weather Relative Humidity
GHR	Global Horizontal Radiation
DHR	Diffuse Horizontal Radiation
WD	Wind Direction
WDR	Weather Daily Rainfall
RGT	Radiation Global Tilted
RDT	Radiation Diffuse Tilted
AP	Active Power

II. Data Description

In this paper, experimental data was collected from a real PV power generation system in DKASC, Alice Springs, Australia [22] and this data are publicly available. The entire dataset contains not only the power (kW) information generated by the PV devices but also other related information. The relevant information and its abbreviations are shown in Table I.

III. Problem Formulation

If a time series contains many kinds of information (e.g., it contains power generation, temperature, humidity, etc.), we call each different kind of information a feature (e.g., there are 10 kinds of information in Table I, i.e., 10 features). Further, when a time series data set has N features, these features are all collected with the same start time, have the same sampling rate, and continue for T time steps. A raw multivariate time series can be defined in the following form:

$$\begin{pmatrix} \boldsymbol{\chi}_{\boldsymbol{a}}^{1} & \boldsymbol{\chi}_{\boldsymbol{a}}^{2} & \boldsymbol{\chi}_{\boldsymbol{a}}^{3} & \cdots & \boldsymbol{\chi}_{\boldsymbol{a}}^{T} \\ \boldsymbol{\chi}_{\boldsymbol{b}}^{1} & \boldsymbol{\chi}_{\boldsymbol{b}}^{2} & \boldsymbol{\chi}_{\boldsymbol{b}}^{3} & \cdots & \boldsymbol{\chi}_{\boldsymbol{b}}^{T} \\ \boldsymbol{\chi}_{\boldsymbol{C}}^{1} & \boldsymbol{\chi}_{\boldsymbol{C}}^{2} & \boldsymbol{\chi}_{\boldsymbol{C}}^{3} & \cdots & \boldsymbol{\chi}_{\boldsymbol{C}}^{T} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \boldsymbol{\chi}_{\boldsymbol{N}}^{1} & \boldsymbol{\chi}_{\boldsymbol{N}}^{2} & \boldsymbol{\chi}_{\boldsymbol{N}}^{3} & \cdots & \boldsymbol{\chi}_{\boldsymbol{N}}^{T} \end{pmatrix} \tag{1}$$

Where the superscript T represents the time step and the subscript N represents that the variable belongs to a different feature. In particular, when $N = 1$, it becomes a univariate time series. Then $\boldsymbol{\chi}_N^T$ represents the value of N features at moment T in the multivariate time series. We can also represent a multivariate time series in the following simple form: $\{\boldsymbol{\chi}^1, \ldots, \boldsymbol{\chi}^t, \ldots, \boldsymbol{\chi}^T\} \in \mathbb{R}^{T \times N}$, where $\boldsymbol{\chi}^t = [\chi_1^t, \ldots, \chi_N^t] \in \mathbb{R}^N$

A time $\bar{\bar{t}}$ is given as a separating marker. The time series before $\bar{\bar{t}}$ can be expressed as $\{\boldsymbol{\chi}^1, \ldots, \boldsymbol{\chi}^{\bar{\bar{t}}-1}\}$ which represents the input to the problem. We use $\{\hat{\boldsymbol{\chi}}^{\bar{\bar{t}}}, \hat{\boldsymbol{\chi}}^{\bar{\bar{t}}+1}, \ldots\}$ to denote the forecasting results made by models. The goal is to make the prediction result $\{\hat{\boldsymbol{\chi}}^{\bar{\bar{t}}}, \hat{\boldsymbol{\chi}}^{\bar{\bar{t}}+1}, \ldots\}$ as close as possible to real values $\{\boldsymbol{\chi}^{\bar{\bar{t}}}, \boldsymbol{\chi}^{\bar{\bar{t}}+1}, \ldots\}$. Also, we denote the length of the input time series $\{\boldsymbol{\chi}^1, \ldots, \boldsymbol{\chi}^{\bar{\bar{t}}-1}\}$ as $\mathcal{T}$, which represents the length of the historical information that the model can "observe". The length of the output time series $\{\hat{\boldsymbol{\chi}}^{\bar{\bar{t}}}, \hat{\boldsymbol{\chi}}^{\bar{\bar{t}}+1}, \ldots\}$ is denoted as $\boldsymbol{\mathcal{F}}$, which represents the length of the series that the model forecasts.

Typically, long-term forecasting refers to forecasting over a period of months or years. However, to the best of our knowledge, there is no precise definition for long-sequence forecasting. Some of this research identify scenarios where $\boldsymbol{\mathcal{F}} \geq \boldsymbol{\mathcal{T}}$ as "long-sequence" [18]. However, another kind of definition is used in this paper, i.e., this forecast is considered as a long-sequence time series forecast when $\boldsymbol{\mathcal{F}} \geq 48$ [5].

It is particularly important to state that, unlike other multivariate time series forecasting tasks, multivariate PV time series forecasting does not require forecasting the values of all future feature dimensions. Thus, the tasks in the PV forecasting task can be defined as receiving $\{\boldsymbol{\chi}^1, \ldots, \boldsymbol{\chi}^{\bar{\bar{t}}-1}\}$ where $\chi^t = [\chi_1^{\bar{\bar{t}}-1}, \ldots, \chi_N^{\bar{\bar{t}}-1}] \in \mathbb{R}^N$, output $\{\hat{\boldsymbol{\chi}}^{\bar{\bar{t}}}, \hat{\boldsymbol{\chi}}^{\bar{\bar{t}}+1}, \ldots, \hat{\boldsymbol{\chi}}^{\bar{\bar{t}}+47}, \ldots\}$, and the output has only one feature dimension.

IV. Methodology

The proposed forecasting model will be described in this section. The model consists of two components, i.e., anomaly processing and forecasting network.

A. Anomaly Processing

Data quality is the cornerstone for building successful models [23]. The main role of the anomaly processing module is to detect and correct singular values in the data, especially the values we eventually have to forecast. We want to use a relatively conservative strategy for data cleaning to ensure that as little information as possible is lost in the process. The method of anomaly data processing is shown in Table II. The following focuses on an Isolation Forest (**iForest**)-based [24] singular value detection and correction algorithm.

PV power generation follows the equation below:

$$\boldsymbol{\mathcal{E}} = \bar{\mathcal{A}} \sigma \bar{\eta} \varrho \tag{2}$$

where, $\boldsymbol{\mathcal{E}}$ stands for output energy (kW), $\bar{\mathcal{A}}$ is the size of the PV panel (m^2), σ represents solar panel yield or efficiency (%), $\bar{\eta}$ is the average solar radiation of the target panel in the time interval (kWh/m^2), and ϱ refers to the performance ratio (%). Next, the **RGT-AP** curve is plotted, as shown in Fig. 2, which shows a linear relationship between **AP** and **RGT**. Therefore we have a hypothesis: for a certain **RGT** value, its corresponding **AP** value should have an upper and lower bound. This hypothesis is based on the fact that, according to (2), the value of **AP** is related to four variables. For the same PV panel, $\bar{\mathcal{A}}$ and σ can be considered as deterministic values. Although the performance ratio ϱ can be affected by many factors, the ϱ of the same facility should be relatively stable. We regard points beyond the upper and lower bounds as singularities.

First, the **RGT-AP** points data were divided into multiple intervals according to the **RGT** values. On each interval, the iForest algorithm is then used. The iForest algorithm is an unsupervised anomaly detection algorithm proposed by Liu et al. [24] The design of the iForest algorithm is based on two assumptions: first, the percentage of anomalous data in all data

Authorized licensed use limited to: Mukesh Patel School of Technology & Engineering. Downloaded on July 21,2026 at 04:29:54 UTC from IEEE Xplore. Restrictions apply.

TABLE II
THE REASONS FOR THE OCCURRENCE OF SINGULAR VALUES, THE PERCENTAGE OF THEM, AND HOW TO PROCESS THEM

Features	Anomalies	Proportion(%)	Possible reasons	Processing method
WD	Out of [0,360]	0.7	Unusual measurement errors	Flooring and capping
WRH	Out of [0,100]	0.8	Unusual measurement errors	Flooring and capping
AP	Less than 0	1.2	Unusual measurement errors	Set to 0
	Detected singular values	1.8	Measurement error/Facility damage	Correction by our method

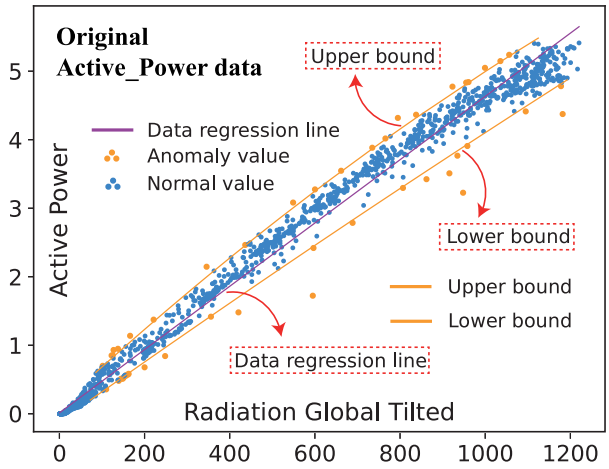


Fig. 2. Anomaly detection results.

should be small; second, the anomalies are the outliers that are more easily isolated. What the algorithm needs to do is to build many isolation trees (iTrees) and then combine them into iForest. The specific method for constructing an iTree is as follows.

1) Randomly selects $\bar{\bar{\psi}}$ points from the data as subsamples and put them into the root node of the tree.
2) A random cut point $\mathcal{P}$ is generated in the current node data - the cut point is generated between the maximum and minimum values of the independent variable dimension of the current node data.
3) A hyperplane is generated with this cut point $\mathcal{P}$. Then the current node data space is divided into 2 subspaces: the data less than $\mathcal{P}$ is taken as the left child of the current node, and the data greater than or equal to $\mathcal{P}$ is taken as the right child of the current node.
4) Recursive steps 2 and 3 in the child nodes, and keep constructing new children until there is only one data in the child node (no more cutting) or the child node has reached the limit height.

After obtaining the specified number of iTrees, the iForest training is finished, and we can then use the generated iForest to evaluate the test data. For a test $\bar{\mathcal{T}}$, we make it traverse each iTree and calculate how many levels of nodes (tree heights) $\bar{\mathcal{T}}$ finally needs to go through to complete the segmentation. Then we can obtain the average height of $\bar{\mathcal{T}}$ in each tree. That is, the average path length over $\bar{\mathcal{T}}$ iTrees. After obtaining the average path length of each test data, we can set a threshold (boundary value), and the test data with an average path length lower than this threshold is the anomaly data.

The RGT-AP data were divided into $\mathcal{N}$ intervals of every 50 according to the value of RGT, and the anomaly detected by the iForest algorithm for each interval is $\{\widetilde{\chi}_1, \widetilde{\chi}_2 \cdots \widetilde{\chi}_n\}$. Where $\widetilde{\chi}$ denotes the anomaly points and satisfies $\boldsymbol{\mathcal{MAX}}_{PL}(\widetilde{\chi}_n) > Threshold$. Then the set of all anomalies can be expressed as:

$$\boldsymbol{\mathcal{U}}_{anomaly} = \bigcup_{i=1}^{\mathcal{N}} \{\widetilde{\chi}_1, \widetilde{\chi}_2 \cdots \widetilde{\chi}_n\}_i \tag{3}$$

$\boldsymbol{\mathcal{MAX}}_{PL}$ denotes the max path length. And the subscript i represents that the outlier is from the i-th interval.

Next, a simple linear regression was performed on all the normal data. The regression yields a function expressed as $\boldsymbol{\mathcal{F}}(x)$. All anomalous data were then divided into two parts:

$$\boldsymbol{\mathcal{U}}_{anomaly} = \begin{cases} \mathcal{U}^{upper}_{anomaly} & \phi^i_n > \boldsymbol{\mathcal{F}}(\widetilde{\chi}^i_n) \\ \mathcal{U}^{lower}_{anomaly} & \phi^i_n \leq \boldsymbol{\mathcal{F}}(\widetilde{\chi}^i_n) \end{cases} \tag{4}$$

In (4), the superscript i denotes the interval to which it belongs, the subscript n denotes the n-th outlier in the interval, and ϕ denotes the true value of the dependent variable at the outlier. Then linear regression was performed for $\mathcal{U}^{upper}_{anomaly}$ and $\mathcal{U}^{lower}_{anomaly}$ separately, and the following results were obtained:

$$\boldsymbol{\mathcal{B}} = \begin{cases} \mathcal{B}_{upper} = \boldsymbol{LR}\left(\mathcal{U}^{upper}_{anomaly}\right) \\ \mathcal{B}_{lower} = \boldsymbol{LR}\left(\mathcal{U}^{lower}_{anomaly}\right) \end{cases} \tag{5}$$

$\boldsymbol{\mathcal{B}}$ represents the decision boundary and LR represents linear regression. Finally, all test data is determined according to the decision boundary, and exceeding the decision boundary is considered a singularity. Decision boundaries, regression lines for all data, singularities, and normal points are all shown in Fig. 2.

Finally, we train a LightGBM [12] $\boldsymbol{\Gamma}(x)$ using all normal points, and the $\boldsymbol{\Gamma}(x)$ can predict ϕ^i_n by $\widetilde{\chi}^i_n$. Then all singularities are corrected with $\boldsymbol{\Gamma}(x)$:

$$\widehat{\phi}^i_n = \boldsymbol{\Gamma}(\widetilde{\chi}^i_n) \tag{6}$$

Where $\widehat{\phi}^i_n$ represents the corrected dependent variable value. And the correction results are shown in Fig. 3.

B. Forecasting Network

After completing the data preprocessing, the forecasting network will be constructed next.

1) Decomposition Module [18]: Many time series analysis methods use a decomposition mechanism to decompose the time series into multiple subseries. Similarly, a decomposition module for multivariate time series is defined in the neural network: For length-T input series $\mathcal{X} \in \mathbb{R}^{T \times N}$, the decomposition

Authorized licensed use limited to: Mukesh Patel School of Technology & Engineering. Downloaded on July 21,2026 at 04:29:54 UTC from IEEE Xplore. Restrictions apply.

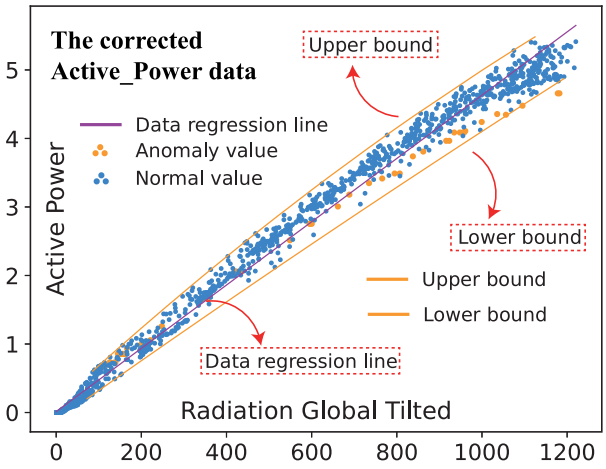


Fig. 3. Singular value correction result.

operation is represented by the following:

$$\begin{aligned}\bar{\tilde{\mathcal{X}}}_{\mathrm{t}} &= \mathbf{AvgPool}\left(\mathrm{Padding}(\mathcal{X})\right)\\ \bar{\tilde{\mathcal{X}}}_{\mathrm{s}} &= \mathcal{X} - \bar{\tilde{\mathcal{X}}}_{\mathrm{t}}\end{aligned} \tag{7}$$

In (7), $\bar{\tilde{\mathcal{X}}}_{\mathrm{s}}, \bar{\tilde{\mathcal{X}}}_{\mathrm{t}} \in \mathbb{R}^{T\times N}$ denote the seasonal part and the trend part. **AvgPool** denotes a moving average with a Padding mechanism to maintain the length of the series. The whole module is represented by $\bar{\tilde{\mathcal{X}}}_{\mathrm{t}} = \mathbf{SeriesDecomp}(\mathcal{X})$. Then the i-th decomposition can be defined as:

$$\mathbf{SeriesDecomp}^{i}\left(\bar{\tilde{\mathcal{X}}}_{\mathrm{t}}^{i-1}\right)) = \bar{\tilde{\mathcal{X}}}_{\mathrm{t}}^{i} \tag{8}$$

The n-th decomposition of the series can be expressed as:

$$\begin{aligned}\mathbf{Decomp}^{n}(\mathcal{X}) = \mathcal{X} \hat{\oplus} \mathbf{SeriesDecomp}^{1}(\mathcal{X}) \hat{\oplus}\\ \mathbf{SeriesDecomp}^{2}\left(\bar{\tilde{\mathcal{X}}}_{\mathrm{t}}^{1}\right) \hat{\oplus} \cdots \hat{\oplus} \mathbf{SeriesDecomp}^{n}\left(\bar{\tilde{\mathcal{X}}}_{\mathrm{t}}^{n-1}\right)\end{aligned} \tag{9}$$

where the superscript i means this is the i-th decomposition and $\hat{\oplus}$ means the vector is concatenated according to the third dimension, i.e., according to the channel. This module will be used in the network.

2) Interval Encoding Module: It should be noted that the network references a network used for computer vision, namely MLP-Mixer [25]. This feature extraction structure is promising and has been used in recommender systems [26] and natural language modeling [27], but to our knowledge, has not been applied to time series forecasting tasks. First, the encoding module receives an input time series $\mathcal{X}$ with shape $\mathcal{H} \times \mathcal{W}$. For illustration purposes, here $\mathcal{H}$ represents the feature dimension of the time series, and $\mathcal{W}$ represents the length of the input series. Next, this time series is split by $\mathcal{W}$ dimensions, i.e., the entire time series is divided into multiple intervals. Then the shape of each divided interval can be expressed as $\mathcal{H} \times \mathcal{W}/n$. In particular, define $\boldsymbol{\mathcal{S}} = \boldsymbol{\mathcal{W}}/\boldsymbol{n}$. Where n denotes the number of intervals. We make it pass through the **SeriesDecomp** module and merge the trend items into the interval as channel information. If multiple decompositions are made, the trend term obtained from the previous decomposition is used as input for the next decomposition. We define the decomposition operation as being performed $\mathcal{Q}$ times. Then the shape of each interval now becomes $\mathcal{H} \times (\mathcal{W}/n) \times (\mathcal{Q}+1)$. For each such interval, make it encoded as a $\mathcal{C}$-dimensional vector through the same fully connected layer. After the above operation, each interval is decomposed and encoded as a $\mathcal{C}$-dim vector and formed into an $\mathcal{M} \in \mathbb{R}^{\mathcal{S}\times\mathcal{C}}$matrix for later use.

The representation of the whole encoding process is as follows.

$$\begin{aligned}\mathbf{Encoding}(\mathcal{X}) &= \mathbf{FC}\left(\diamond\left(\mathbf{Decomp}^{\mathcal{Q}}(\mathcal{X})\right)\right)\\ &= \mathbf{FC}\left(\hat{\tilde{\mathcal{X}}}_1 \bar{\oplus} \hat{\tilde{\mathcal{X}}}_2 \bar{\oplus} \cdots \bar{\oplus} \hat{\tilde{\mathcal{X}}}_k\right)\\ &= \mathbf{FC}(\hat{\tilde{\mathcal{X}}}_1) \bar{\oplus} \mathbf{FC}(\hat{\tilde{\mathcal{X}}}_2) \bar{\oplus} \cdots \bar{\oplus} \mathbf{FC}(\hat{\tilde{\mathcal{X}}}_k)\\ &= \boldsymbol{\mathcal{V}}_1 \bar{\oplus} \boldsymbol{\mathcal{V}}_2 \bar{\oplus} \cdots \bar{\oplus} \boldsymbol{\mathcal{V}}_k\\ &= \overline{\mathcal{X}}\end{aligned}$$
$$\mathbf{FC}(\hat{\tilde{\mathcal{X}}}_i) = \boldsymbol{\mathcal{W}}\hat{\tilde{\mathcal{X}}}_k + \boldsymbol{\mathcal{B}} = \boldsymbol{\mathcal{V}}_k \tag{10}$$

where **FC** stands for fully connected, $\diamond$ stands for split operation along $\boldsymbol{\mathcal{W}}$ dimensions, $\hat{\tilde{\mathcal{X}}}_k$ denotes the interval after decomposition and split, $\bar{\oplus}$ stands for vector concatenation by $\boldsymbol{\mathcal{W}}$ dimensions, $\boldsymbol{\mathcal{V}}_k$ is the result of each interval after **FC**, and $\{k \in \mathbb{N}^*, 1 \le k \le \boldsymbol{\mathcal{S}}\}$. The final encoded result is denoted by $\overline{\mathcal{X}}$.

3) Mixing Module: First, let the encoded vector pass through the LayerNormal layer [28]. Then let it pass through the mixer-layer. Each mixer-layer consists of two parts.

The first part is the **Interval-Mixer**, which allows communication between intervals. More precisely, it will map the columns of the matrix composed of the encoded vectors. This process maps the columns to hidden variables and then from the hidden variables to columns of the same dimensions ($\mathbb{R}^{\mathcal{S}} \to \mathbb{R}^{\mathcal{S}}$).

The second part is the **Channel-Mixer**, which allows channels to communicate with each other. The whole operation is performed on the rows of $\mathcal{M}$. Moreover, it is an $\mathbb{R}^{\mathcal{C}} \to \mathbb{R}^{\mathcal{C}}$ mapping.

Both of the above mixing operations are implemented through the **MLP-Block**. Each **MLP-Block** contains two Fully-connected layers and one non-linear activation layer. For the nonlinear activation function, Swish is used, which can be expressed as:

$$\mathbf{Swish}(\mathcal{X}) = \mathcal{X}\,\mathbf{sigmoid}(\beta\mathcal{X}) = \frac{\mathcal{X}}{1+e^{-\beta\mathcal{X}}} \tag{11}$$

where β is either constant or a trainable parameter depending on the model. Also, the mixer-layer can be represented in the following form:

$$\begin{aligned}\boldsymbol{\Phi}_{*,i} &= \overline{\mathcal{X}}_{*,i} + \mathcal{W}_2\rho\left(\mathcal{W}_1\,\mathbf{LN}(\overline{\mathcal{X}})_{*,i}\right), \text{ for } i = 1\ldots\mathcal{C},\\ \mathbf{Y}_{j,*} &= \boldsymbol{\Phi}_{j,*} + \mathcal{W}_4\rho\left(\mathcal{W}_3\,\mathbf{LN}(\boldsymbol{\Phi})_{j,*}\right), \text{ for } j = 1\ldots\mathcal{S}\end{aligned} \tag{12}$$

Authorized licensed use limited to: Mukesh Patel School of Technology & Engineering. Downloaded on July 21,2026 at 04:29:54 UTC from IEEE Xplore. Restrictions apply.

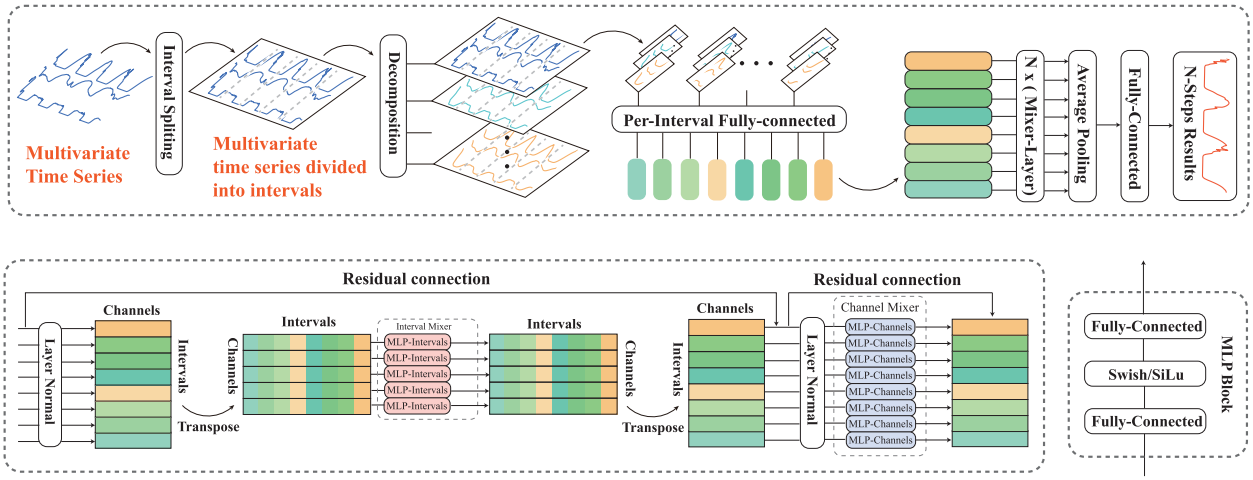


Fig. 4. The upper picture shows the workflow of **Solar-Mixer** Net. The bottom picture shows how Mixer or mixer-Layer works and the composition of the basic MLP module.

In (12), $\overline{\mathcal{X}}$ represents the input, **LN** denotes the LayerNormal operation, $\mathcal{W}$ stands for weight, ρ for nonlinear activation, $\mathbf{\Phi}_{j,*}$ represents the result of Interval mixing, $\mathbf{Y}_{j,*}$ for mixer-layer output, and the additive operation represents the residual connection. In the mixer-layer, all columns share the same **Interval-Mixer**, and all rows share the same **Channel-Mixer**. The model structure, mixer-layer, and the **MLP-Block** structure are shown in Fig. 4.

4) MLP-Block and Its Back Propagation Process: For an MLP module with loss function MSE:

$$\mathcal{C} = \left(a^L - y\right)^2 \tag{13}$$

we let the output of the $l-$th layer ($l = 1, 2, \cdots L$) before going through the activation function be z^l, the output after activation be a^l and $\boldsymbol{\sigma}$ denote the activation function. Also defined, the activated output of the previous layer is used as the input of the next layer. Then the two outputs of layer l can be written as:

$$\begin{aligned} z^l &= \mathcal{W}^l a^{l-1} + \mathcal{B}^l \\ a^l &= \boldsymbol{\sigma}\left(z^l\right) \end{aligned} \tag{14}$$

This is also known as the forward propagation process of MLP. To facilitate further computational derivation, as well as to avoid repeated calculations, we introduce an intermediate quantity δ^l, which we call the δ-error at layer l, denoting the partial derivative of the error function with respect to the output value of the neural network before an activation at layer l, i.e., $\delta^l = \frac{\partial C}{\partial z^l}$.

$$\begin{aligned} \frac{\partial \mathcal{C}}{\partial a^l} &= 2\left(a^l - y\right) \\ \frac{\partial a^l}{\partial z^l} &= \boldsymbol{\sigma}'\left(z^l\right) \\ \delta^l = \frac{\partial \mathcal{C}}{\partial z^l} &= 2\left(a^l - y\right) \odot \sigma'\left(z^l\right) \end{aligned} \tag{15}$$

$\odot$ denotes Hadmard product. Meanwhile, the derivative of the loss function with respect to each parameter of the current layer is:

$$\begin{aligned} \frac{\partial \mathcal{C}}{\partial \mathcal{W}^l} &= \frac{\partial \mathcal{C}}{\partial z^l}\frac{\partial z^l}{\partial \mathcal{W}^l} = \delta^l\left(a^{l-1}\right)^T \\ \frac{\partial \mathcal{C}}{\partial \mathcal{B}^l} &= \frac{\partial \mathcal{C}}{\partial z^l}\frac{\partial z^l}{\partial \mathcal{B}^l} = \delta^l \odot \mathbf{1} = \delta^l \end{aligned} \tag{16}$$

Then for the δ of the $l-1$ layer, it can be expressed as follows.

$$\begin{aligned} \delta^{l-1} &= \frac{\partial \mathcal{C}}{\partial z^{l-1}} = \frac{\partial \mathcal{C}}{\partial z^l}\frac{\partial z^l}{\partial z^{l-1}} = \delta^l \frac{\partial z^l}{\partial z^{l-1}} \\ z^l &= \mathcal{W}^l a^{l-1} + \mathcal{B}^l = \mathcal{W}^l \sigma\left(z^{l-1}\right) + \mathcal{B}^l \\ \delta^{l-1} &= \left(\mathcal{W}^l\right)^T \delta^l \odot \sigma'\left(z^{l-1}\right) \end{aligned} \tag{17}$$

Since we previously calculated the $delta$-error of the last layer δ^l, through the above equation, we can find the δ-error of δ^{l-1}, δ^{l-2} in turn all the way to the second layer δ^2. The gradient of each layer is:

$$\begin{aligned} \frac{\partial \mathcal{C}}{\partial \mathcal{W}^l} &= \frac{\partial \mathcal{C}}{\partial z^l}\frac{\partial z^l}{\partial \mathcal{W}^l} = \delta^l\left(a^{l-1}\right)^T \\ \frac{\partial \mathcal{C}}{\partial \mathcal{B}^l} &= \frac{\partial \mathcal{C}}{\partial z^l}\frac{\partial z^l}{\partial \mathcal{B}^l} = \delta^l \odot 1 = \delta^l \end{aligned} \tag{18}$$

The parameter update formula is:

$$\begin{aligned} \mathcal{W}^l &= \mathcal{W}^l - \eta\frac{\partial \mathcal{C}}{\partial \mathcal{W}^l} \\ \mathcal{B}^l &= \mathcal{B}^l - \eta\frac{\partial \mathcal{C}}{\partial \mathcal{B}^l} \end{aligned} \tag{19}$$

η stands for learning rate.

5) Structure Analysis: Similar to the original MLP-Mixer structure, this network can also be seen as a very special kind of convolutional neural network. In contrast to the vision task, we consider that time series forecasting does not require convolution, a particularly intense inductive bias. However, to be able to receive inputs from longer time series, the model uses a method of splitting the time series into multiple intervals and

Authorized licensed use limited to: Mukesh Patel School of Technology & Engineering. Downloaded on July 21,2026 at 04:29:54 UTC from IEEE Xplore. Restrictions apply.

then sharing a common encoder. This actually also introduces a priori information, which is that the data within each interval always have a strong relationship. The benefit of this is also obvious; no matter how long the series input is, no additional model parameters are added during the encoding phase, allowing the model to receive extremely long series inputs.

Also, as the length of the input sequence increases, the complexity of the mixer-layer increases. But this increase is more gradual (compared to Self-Attention). Another reason for using the mixer-layer as a feature extraction network is that the mixer-layer receives inputs of the same size, i.e., it is "isotropic," which is consistent with **Transformer** and deep RNNs. The other difference with the **Transformer** structure is that Mixer does not require positional embedding since Interval-Mixing MLPs are sensitive to interval order. For the final decoding process, we only use a fully-connected layer to directly output the n-step prediction results without using a complex decoder structure.

V. Experiment Details and Settings

In addition to the data mentioned in Section II, we also conducted experiments on other sites that have the same feature dimensions as the data in Section II. The other two sets of data are from 37, DKASC, Alice Springs, Australia, and 35, DKASC, Alice Springs, Australia.

A. Solar Data

For Site 38, select the interval from 6:30 AM on February 17, 2021, to 2:30 PM on April 25, 2021 (site 38 interval 1) and from 10:30 AM on May 1, 2021, to 4:00 AM on July 17, 2021 (site 38 interval 2). For site 37, the interval from 11:00 on August 27, 2020, to 14:30 on November 20, 2020 (site 37 interval 1) was chosen for the experiment. For site 35, the interval from 9:00 on September 24, 2015, to 9:30 on December 13, 2015 (site 35 interval 1) was chosen for the experiment. The time granularity of the original data is 15 minutes, which is changed here to half an hour by resampling.

All intervals are strictly divided according to **70%** of the training set, **15%** of the validation set, and **15%** of the test set. The training set, validation set, and test set are to ensure that there is **no time leakage**.

B. Evaluation Metrics

The model prediction effectiveness is measured by two metrics, namely the Mean Absolute Error (**MAE**), Mean Squared Error (**MSE**), and coefficient of determination (R^2). For single-step time series forecasting, define:

$$\mathbf{MSE}_{1-step} = \frac{1}{n}\sum_{i=1}^{n}\left(Y_i - \hat{Y}_i\right)^2 \tag{20}$$

$$\mathbf{MAE}_{1-step} = \frac{1}{n}\sum_{i=1}^{n}\left|Y_i - \hat{Y}_i\right| \tag{21}$$

TABLE III
Model Hyperparameter Settings

Network	MLP/LSTM/Seq-to-Seq/MLP-Mixer	**Solar-Mixer**
Optimizer	AdamW	AdamW
Learning Rate	5e-3	5e-3
Weight Decay	0.0001	0.0001
Schedule	Cosine decay	Cosine decay
Loss Function	MSE	Huber

Network	MLP-Mixer	**Solar-Mixer**
Interval/Patch Size	$48/96 \times NF$	$48/96 \times NF$
Encode Dim	512	512
Mixer-layer Num	2	2/1
Interval/Patch Dim	256	96
Channel Dim	256	96
Decomposition Module	False	True
Anomaly Detection	Flase	True
Activation Function	GELU	Swish

NF: Number of features
The blue text is the parameter used in the univariate prediction

$$\mathbf{R}^2_{1-step} = 1 - \frac{\sum_{i=1}^{n}\left(Y_i - \hat{Y}_i\right)^2}{\sum_{i=1}^{n}\left(Y_i - \bar{Y}\right)^2} \tag{22}$$

where Y_i denotes the true value, $\hat{Y}_i$ denotes the model prediction and $\bar{Y} = \frac{1}{n}\sum_{i=1}^{n} Y_i$. Further, for a multi-step time series forecasting, it is possible to define:

$$\mathbf{MSE}_{n-step} = \frac{\frac{1}{n}\sum_{i=1}^{n}\left(Y_i - \hat{Y}_i\right)^2}{m} \tag{23}$$

$$\mathbf{MAE}_{n-step} = \frac{\frac{1}{n}\sum_{i=1}^{n}\left|Y_i - \hat{Y}_i\right|}{m} \tag{24}$$

$$\mathbf{R}^2_{n-step} = \frac{1 - \frac{\sum_{i=1}^{n}\left(Y_i - \hat{Y}_i\right)^2}{\sum_{i=1}^{n}\left(Y_i - \bar{Y}\right)^2}}{m} \tag{25}$$

m indicates the total number of test intervals, i.e., we average the results of all multi-step predictions.

C. Experimental Design

In the data of Site 38, we perform ablation experiments on the model to verify the effectiveness of the anomaly detection and decomposition modules. On the rest of the data, we compare the proposed model with commonly used time series forecasting models, including the existing **SOTA** models for time series forecasting. The compared models are: **MLP**, **MLP-Mixer**, **LSTM**, LSTM-based **Seq-to-Seq**, **LightGBM**, **Informer**, and **FEDformer**. In addition, to more sufficiently illustrate the effectiveness of the proposed model, the univariate time series forecasting comparison experiment is added to compare the above model along with the classical model of time series forecasting, **ARIMA**. In Site 37 interval 1, we compare the effect of different input lengths on each model.

D. Hyperparameter Setting

In Table III, the hyperparameter settings of the MLP, LSTM, Seq-to-Seq, MLP-Mixer and **Solar-Mixer** (ours) models are

Authorized licensed use limited to: Mukesh Patel School of Technology & Engineering. Downloaded on July 21,2026 at 04:29:54 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
ABLATION STUDY RESULTS

Methods	Metric	Forecasting Steps			
		96	192	384	768
MLP	MSE	0.1772	0.1879	0.2570	0.1535
	MAE	0.2025	0.2182	0.2644	0.2172
	R^2	0.9521	0.9489	0.9292	0.9579
Mixer	MSE	0.1971	0.1950	0.3569	0.1601
	MAE	0.2185	0.2201	0.2661	0.2372
	R^2	0.9466	0.9470	0.9025	0.9560
MixerRE	MSE	0.1685	0.1720	0.1973	0.1232
	MAE	0.1763	0.1811	0.1997	0.1897
	R^2	0.9544	0.9532	0.9458	0.9660
MixerRE+DE	MSE	**0.1676**	0.1683	0.1737	0.1202
	MAE	**0.1587**	0.1729	**0.1825**	0.1762
	R^2	**0.9546**	0.9542	0.9522	0.9668
MixerRE+AD	MSE	0.1788	0.1704	0.1834	0.1278
	MAE	0.2025	0.1759	0.1973	0.2042
	R^2	0.9516	0.9537	0.9496	0.9648
Solar-Mixer-in	MSE	0.1829	0.1819	0.1941	0.1328
	MAE	0.1703	0.1969	0.2033	0.2139
	R^2	0.9505	0.9505	0.9465	0.9635
Solar-Mixer-ch	MSE	0.2205	0.2281	0.2063	0.1789
	MAE	0.2223	0.2389	0.1962	0.1986
	R^2	0.9404	0.9379	0.9434	0.9505
Solar-Mixer	MSE	0.1746	**0.1660**	**0.1733**	**0.1188**
	MAE	0.1778	**0.1648**	0.1895	**0.1721**
	R^2	0.9527	**0.9548**	**0.9523**	**0.9672**

Mixer: MLP-Mixer **MixerRE**: Redesigned MLP-Mixer
DE: Decomposition Module **AD**: Anomaly detection Module
Solar-Mixer-in: Solar-Mixer using Interval-mixer only
Solar-Mixer-ch: Solar-Mixer using Channel-mixer only
Solar-Mixer: MixerRE+DE+AD

presented. *Interval Size* refers to the length of each interval, i.e., the $\mathcal{S}$ variable in Section IV-B2, and *Encode Dim* refers to the vector dimension obtained by encoding, i.e., $\mathcal{C}$ in Section IV-B2. *Channel Dim* and *Interval Dim* refer to the dimension of the hidden variable in the mixer-layer. And the hyperparameter settings of the other models based on the self-attention mechanism are the same in ***FEDformer*** [19] and it's open-source code.

VI. EXPERIMENT RESULTS

A. Ablation Studies

In the ablation study, the experiment will research the following.

1) The impact of anomaly detection and correction on the model.
2) The impact of the time series decomposition module on the model.
3) Effectiveness of improvements for MLP-Mixer architecture.
4) Whether the two Mixing methods can be used individually to achieve the best results.

The results of the ablation study are presented in Table IV. In the experiments, it can be observed that the tuning and redesign of the basic MLP-Mixer are positive and brings an improvement in all output lengths. Also, the addition of the decomposition module is effective and works well among the various metrics for all output lengths. For the anomaly detection and correction module, it is positive for improving all metrics most of the time. However, there are times when this module can have a negative effect on the model, especially when the output length is relatively short. We believe that this is because the use of anomaly detection and correction makes the model focus more on trend information, which leads to the loss of some other features that are important for relatively short-term forecasting. When the decomposition module is added together with the anomaly detection module, most of the time models have the best prediction results at different output lengths. At the same time, the addition of the decomposition module can also alleviate the relatively short-term information loss caused by the anomaly detection module to some extent.

From the experimental results, it can be observed that when $\mathcal{O} = 96$ neither **Solar-Mixer-in** [$\mathbf{R}^2_{s38i1}$(**SM-in**) =0.9505] nor **Solar-Mixer-ch** [$\mathbf{R}^2_{s38i1}$(**SM-ch**) =0.9404] with only a single mixing method can achieve the best results for various prediction lengths. The prediction results of **Solar-Mixer-ch** using only channel mixer are even inferior to the original **Mixer** [$\mathbf{R}^2_{s38i1}$(**Mixer**) =0.9466] without any changes. For this result, we argue that not using the **Channel-Mixer** results in the inability of features to communicate with each other, which in turn leads to a lack of cross-channel correlation in the representation of the hidden space and therefore leads to a decrease in model performance. The decrease in forecasting ability due to not using **Interval-Mixers** is more significant in the experiments. Using only **Channel-Mixers** makes it difficult for the model to capture the correlation between intervals, which leads to more serious degradation in prediction performance. Related studies also support our hypothesis from another perspective [26], [29]. Based on the above facts, we believe that a single mixing method is not sufficient for the whole model to extract enough information for prediction, and both mixing methods need to be used simultaneously.

B. Multivariate Long-Sequence Photovoltaic Time Series Forecasting Results

In Table V, the multivariate prediction results of the proposed and competing models for multiple sites and multiple time intervals are shown. In most cases, the proposed model has the best performance. Even against the most state-of-the-art long-sequence time series forecasting models, the proposed model can still beat them on most metrics. More specifically, on the $\mathbf{R}^2$ metric, the proposed model achieves the best prediction results on all forecast lengths, all stations, and intervals. As an example, when $\mathcal{O} = 768$, [$\mathbf{R}^2_{s38i1}$(**SM**) =0.9016] has a 461% improvement over [$\mathbf{R}^2_{s38i1}$(**Informer**) =0.1605]. For the MSE and MAE metrics, the proposed model also achieves the best results on these two metrics by more than 71% (Number of best results: **23**; Total number of results: **32**) of the cases. At the same time, we point out that the state-of-the-art model has a high training overhead and high equipment requirements, which will be analyzed in detail in Section VI-D.

Authorized licensed use limited to: Mukesh Patel School of Technology & Engineering. Downloaded on July 21,2026 at 04:29:54 UTC from IEEE Xplore. Restrictions apply.

TABLE V
MULTIVARIATE TIME SERIES FORECASTING EXPERIMENTAL RESULTS FOR EACH MODEL AT DIFFERENT STATIONS AND AT DIFFERENT TIME INTERVALS

Models	Metrics	Site 38 Interval 1				Site 38 Interval 2				Site 37 Interval 1				Site 35 Interval 1			
		96	192	384	768	96	192	384	768	96	192	384	768	96	192	384	768
Solar-Mixer	MSE	0.1746	0.1660	**0.1733**	**0.1188**	**0.1365**	**0.1311**	**0.1893**	**0.2964**	0.1573	**0.1219**	**0.0994**	**0.1482**	0.3929	0.4292	0.3858	0.2720
	MAE	0.1778	**0.1648**	**0.1895**	**0.1721**	**0.1852**	0.1845	**0.2698**	**0.3246**	**0.2032**	**0.1786**	**0.1743**	**0.1970**	**0.3280**	**0.3343**	**0.3049**	**0.2551**
	R^2	**0.9527**	**0.9548**	**0.9523**	**0.9672**	**0.9532**	**0.9554**	**0.9362**	**0.9016**	**0.9556**	**0.9661**	**0.9726**	**0.9586**	**0.8417**	**0.8278**	**0.8491**	**0.8964**
Mixer	MSE	0.1971	0.1950	0.3569	0.1601	0.1546	0.1377	0.2226	0.3797	0.1649	0.1443	0.1864	0.1824	0.4099	0.4382	0.3964	0.2873
	MAE	0.2185	0.2201	0.2661	0.2372	0.2677	**0.1815**	0.2880	0.3811	0.2317	0.2137	0.2863	0.2484	0.3367	0.3592	0.3397	0.3086
	R^2	0.9466	0.9470	0.9025	0.9560	0.9471	0.9533	0.9251	0.8739	0.9535	0.9599	0.9497	0.9495	0.8348	0.8242	0.8449	0.8920
MLP	MSE	0.1772	0.1879	0.2570	0.1535	0.1555	0.1624	0.2038	0.3116	0.2017	0.1348	0.1530	0.1550	0.4178	0.4412	0.3875	0.2846
	MAE	0.2025	0.2182	0.2644	0.2172	0.2052	0.2182	0.2906	0.3471	0.2644	0.1945	0.2404	0.2127	0.3352	0.3625	0.3085	0.2710
	R^2	0.9521	0.9489	0.9292	0.9579	0.9467	0.945	0.9313	0.8971	0.9431	0.9625	0.9584	0.9568	0.8316	0.8230	0.8486	0.8923
LSTM	MSE	0.2038	0.2214	0.3576	0.2228	0.2356	0.158	0.2254	0.3678	0.1830	0.1770	0.3319	0.2167	0.4949	0.5303	0.4287	0.2904
	MAE	0.2002	0.2508	0.2995	0.2827	0.2506	0.2311	0.2909	0.3594	0.2310	0.2470	0.3670	0.2611	0.3567	0.4110	0.3685	0.2811
	R^2	0.9448	0.9398	0.9018	0.9386	0.9193	0.9464	0.9241	0.8788	0.9484	0.9509	0.9110	0.9401	0.8005	0.7875	0.8342	0.8911
FEDformer	MSE	0.2482	0.2491	0.1870	0.2371	0.3440	0.3163	0.8875	0.6925	0.5593	0.6024	0.6526	0.5205	6.4529	6.7716	7.0468	**0.1921**
	MAE	0.3384	0.3445	0.3221	0.3878	0.3980	0.3764	0.4422	0.5715	0.5026	0.5164	0.5274	0.4378	0.9651	0.9271	0.8340	0.3015
	R^2	0.7683	0.7669	0.8239	0.7759	0.6838	0.7116	0.1862	0.3693	0.3672	0.3188	0.2856	0.4551	-7.5676	-7.8876	-7.6937	0.7780
Informer	MSE	**0.1327**	**0.1657**	0.3114	1.0163	0.2071	0.1885	0.2640	0.9217	**0.1467**	0.1558	0.1967	0.5571	**0.3201**	**0.4113**	**0.3332**	0.3537
	MAE	0.2536	0.3034	0.4536	0.8522	0.3132	0.3103	0.3770	0.7433	0.2504	0.2813	0.3208	0.6087	0.3929	0.4579	0.4269	0.4696
	R^2	0.8760	0.8449	0.7067	0.0398	0.8096	0.8281	0.7578	0.1605	0.8340	0.8237	0.7846	0.4168	0.5749	0.4600	0.5888	0.5913
LightGBM	MSE	0.2031	0.2835	0.3847	0.3320	0.2228	0.2352	0.3749	0.8023	0.3470	0.4127	0.3009	0.4877	0.4005	0.4705	0.4475	0.3824
	MAE	**0.1695**	0.2349	0.2910	0.2698	0.2295	0.2338	0.3148	0.4778	0.2981	0.3387	0.2844	0.3537	0.4649	0.3463	0.3358	0.3168
	R^2	0.9450	0.9231	0.8943	0.9088	0.9240	0.9202	0.8732	0.7346	0.9020	0.8850	0.9183	0.8613	0.8386	0.8108	0.8253	0.8564
Seq-to-Seq	MSE	0.3997	0.3448	0.4037	0.2712	0.3746	0.4510	0.6468	0.6950	0.2551	0.2838	0.3197	0.3978	0.6696	0.6205	0.5426	0.3864
	MAE	0.4212	0.3931	0.3760	0.3012	0.4040	0.3976	0.5107	0.5351	0.3165	0.3632	0.3648	0.3965	0.5556	0.5349	0.4867	0.3845
	R^2	0.8918	0.9064	0.8892	0.9252	0.8719	0.8480	0.7824	0.7704	0.9280	0.9214	0.9140	0.8911	0.7300	0.6529	0.7878	0.8586

The input to all multivariate time series forecasting models is $I = 96$ and the output is of length $\mathcal{O} \in \{96, 192, 384, 768\}$. The lower the MSE and MAE the better, and the higher the R^2 the better. The best results will be bolded and underlined.

C. Univariate Long-Sequence Photovoltaic Time Series Forecasting Results

In Table VI, the results of the proposed model and the comparison model for univariate time series forecasting are shown. Compared to multivariate time forecasting, the advantage of the proposed model on univariate time series forecasting is not as great. However, the proposed model still achieves the most optimal results in three metrics (Number of best results: **21**; Total number of results: **36**).

In this section, **ARIMA** is the most competitive opponent of the proposed model, and we have some discussion of this. For the R^2 metric analysis, when $\mathcal{O}$=384, the proposed model achieves the best results [$\mathbf{R}^2_{s37i1}$(**Solar-Mixer**) =0.9686], it has at most a 1.9% improvement over the ARIMA model [$\mathbf{R}^2_{s37i1}$(**ARIMA**) =0.9502]. The MSE and MAE of the proposed model also show a 39% [$\mathbf{MSE}_{s37i1}$(**ARIMA**) =0.1817 $\rightarrow$ $\mathbf{MSE}_{s37i1}$(**Solar-Mixer**) =0.1106] and 32% [$\mathbf{MAE}_{s37i1}$(**ARIMA**) =0.2379 $\rightarrow$ $\mathbf{MAE}_{s37i1}$(**Solar-Mixer**) =0.1612] reduction, respectively, compared to ARIMA for the same input and output lengths. However, when analyzing the only time when ARIMA achieves the best results in all three metrics, it can be found that the proposed model [$\mathbf{R}^2_{s38i2}$(**Solar-Mixer**) =0.9473] has only a 0.8% difference in the R^2 metric compared to ARIMA [$\mathbf{R}^2_{s38i2}$(**ARIMA**) =0.9518]. Also, the difference between its MSE and MAE is only 15% [$\mathbf{MSE}_{s37i1}$(**ARIMA**) =0.1340 $\rightarrow$ $\mathbf{MSE}_{s37i1}$(**Solar-Mixer**) =0.1571] and 17% [$\mathbf{MAE}_{s37i1}$(**ARIMA**) =0.1939 $\rightarrow$ $\mathbf{MAE}_{s37i1}$(**Solar-Mixer**) =0.2342].

From the experimental results, it is clear that ARIMA is undoubtedly a strong competitor and that ARIMA has a lesser computational overhead. But in any case, the proposed model still achieves the most optimal results in univariate time series forecasting and the difference with ARIMA is not significant even in scenarios where the metrics are not as good as ARIMA.

D. Model Training Cost Analysis

All experiments are performed on devices with a CPU: AMD Ryzen 5600x, GPU: Nvidia RTX 2070 s, and Python 3.8, Pytorch 1.8.1, and Cuda 10.1.

Authorized licensed use limited to: Mukesh Patel School of Technology & Engineering. Downloaded on July 21,2026 at 04:29:54 UTC from IEEE Xplore. Restrictions apply.

TABLE VI
UNIVARIATE TIME SERIES FORECASTING EXPERIMENTAL RESULTS FOR EACH MODEL AT DIFFERENT STATIONS AND AT DIFFERENT TIME INTERVALS

Models	Metrics	Site 38 Interval 1				Site 38 Interval 2				Site 37 Interval 1			
		96	192	384	768	96	192	384	768	96	192	384	768
Solar-Mixer	MSE	0.0786	**0.1055**	0.1306	**0.1170**	0.1251	**0.1127**	0.1616	0.1571	0.1881	0.1578	0.1106	**0.1105**
	MAE	**0.1710**	**0.1419**	**0.1732**	**0.1585**	**0.1720**	0.1920	0.2325	0.2342	0.1816	**0.1770**	**0.1612**	**0.1619**
	R^2	**0.9795**	**0.9716**	**0.9642**	**0.9674**	0.9566	**0.9617**	0.9456	0.9473	**0.9450**	**0.9546**	**0.9686**	**0.9687**
Mixer	MSE	0.0872	0.1163	0.2414	0.1943	0.1748	0.2013	0.2278	0.2590	0.1994	0.1736	0.1231	0.1412
	MAE	0.1903	0.1689	0.3226	0.2579	0.2457	0.2934	0.3061	0.3193	0.2396	0.2147	0.1860	0.2288
	R^2	0.9771	0.9687	0.9342	0.9462	0.9396	0.9319	0.9234	0.9138	0.9417	0.9503	0.9653	0.9607
MLP	MSE	0.1278	0.1279	0.1997	0.1371	0.2333	0.2335	0.2487	0.2736	0.2146	0.1896	0.1327	0.1409
	MAE	0.2130	0.1801	0.2756	0.1969	0.2937	0.3022	0.3235	0.3202	0.2614	0.2449	0.1909	0.2245
	R^2	0.9665	0.9655	0.9456	0.9619	0.9194	0.9211	0.9165	0.9091	0.9373	0.9460	0.9629	0.9609
LSTM	MSE	0.1951	0.1402	0.1875	0.1388	0.1383	0.1724	0.2223	0.2558	0.2144	0.1946	0.1323	0.2758
	MAE	0.2433	0.2278	0.2540	0.2085	0.2085	0.2454	0.2769	0.3025	0.1994	0.2713	0.2249	0.3497
	R^2	0.9489	0.9623	0.9488	0.9614	0.9521	0.9417	0.9252	0.9149	0.9373	0.9444	0.9629	0.9244
Seq-to-Seq	MSE	0.1664	0.2799	0.2485	0.1488	0.1820	0.1925	0.2609	0.2943	0.2020	0.1836	0.1535	0.2027
	MAE	0.2428	0.2611	0.2740	0.1939	0.2489	0.2576	0.3020	0.3252	0.2155	0.2399	0.2318	0.2611
	R^2	0.9564	0.9251	0.9325	0.9587	0.9371	0.9351	0.9125	0.9019	0.9409	0.9474	0.9578	0.9444
FEDformer	MSE	**0.0486**	0.2082	**0.1164**	0.4818	**0.0643**	0.1377	**0.1198**	0.3197	**0.0536**	**0.1111**	**0.0901**	0.1853
	MAE	0.1766	0.3671	0.2405	0.5560	0.1724	0.2829	0.2656	0.3846	**0.1530**	0.2482	0.2057	0.2948
	R^2	0.9561	0.8101	0.8904	0.5409	0.9386	0.8723	0.8890	0.7066	0.9394	0.8744	0.9001	0.8027
Informer	MSE	0.9149	0.9427	1.0059	1.0056	0.8797	1.0237	1.0177	1.0422	0.6182	0.6903	0.8492	0.8216
	MAE	0.7458	0.7324	0.8120	0.8578	0.7034	0.7636	0.8809	0.8420	0.6434	0.7131	0.7634	0.7324
	R^2	0.1755	0.1407	0.0538	0.0447	0.1621	0.0513	0.0576	0.0439	0.3024	0.2198	0.0592	0.1232
LightGBM	MSE	0.3785	0.4642	0.4511	0.4682	0.4581	0.5633	0.7512	0.9067	0.3350	0.3933	0.3635	0.8658
	MAE	0.4642	0.5069	0.5053	0.4990	0.4765	0.5079	0.5731	0.6224	0.4372	0.4617	0.4489	0.5625
	R^2	0.9010	0.8768	0.8777	0.8712	0.8418	0.8089	0.7479	0.6994	0.9019	0.8879	0.9009	0.7635
ARIMA	MSE	0.1347	0.1351	0.1358	0.1400	0.1187	0.1220	0.1287	**0.1340**	0.1971	0.1936	0.1817	0.1776
	MAE	0.2144	0.2147	0.2153	0.2175	0.1869	**0.1883**	**0.1912**	**0.1939**	0.2452	0.2431	0.2379	0.2353
	R^2	0.9637	0.9637	0.9638	0.9622	**0.9590**	0.9582	**0.9552**	**0.9518**	0.9444	0.9460	0.9502	0.9516

The input to all univariate time series forecasting models is $I = 768$ and the output is of length $\mathcal{O} \in \{96, 192, 384, 768\}$. The lower the MSE and MAE the better, and the higher the R^2 the better. The best results will be bolded and underlined.

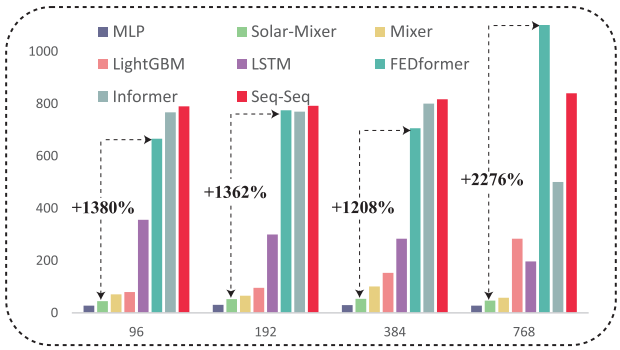


Fig. 5. Training and inference overhead for each model at $I = 96$ and $\mathcal{O} \in \{96, 192, 384, 768\}$.

Here we compare the training and prediction times for all models. This time includes the data preprocessing process, the model training process, and the model prediction process. The comparison results are shown in Fig. 5 . It is obvious from the figure that although the **SOTA** model has advantages in MSE for some sites and intervals, its training cost is high and the overall time even reaches **227** times that of the proposed model. Such computational overhead is not conducive to edge-side deployment and is also not eco-friendly.

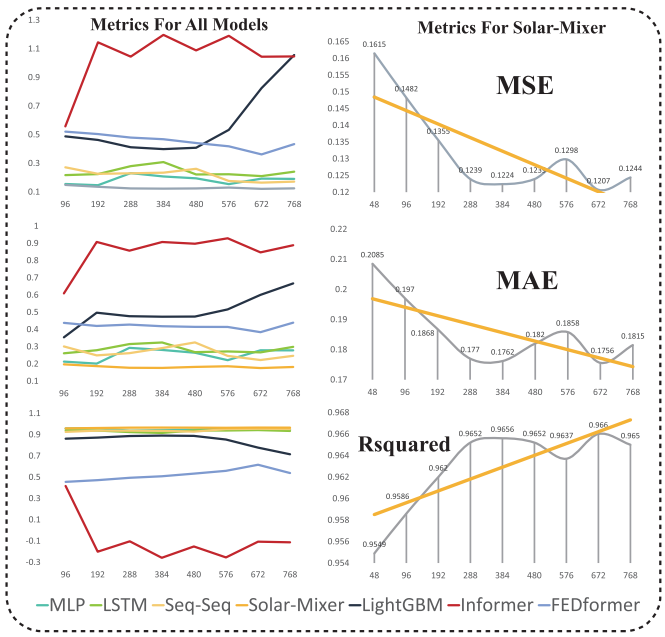


Fig. 6. The right three graphs show the variation of each metric when **Solar-Mixer** receives different lengths of input. The orange line on these graphs is the trend line of the change of each metric. Three graphs on the left side compare the metrics of each model when receiving different input lengths.

Authorized licensed use limited to: Mukesh Patel School of Technology & Engineering. Downloaded on July 21,2026 at 04:29:54 UTC from IEEE Xplore. Restrictions apply.

TABLE VII
COMPARISON OF METRICS FOR EACH MODEL FOR DIFFERENT INPUT LENGTHS, $I = \{96, 192, 288, 384, 480, 576, 672, 768\}$, $\mathcal{O} = 768$

Input Length	96				192				288				384			
Metrics	MSE	MAE	R2	Time	MSE	MAE	R2	Time	MSE	MAE	R2	Time	MSE	MAE	R2	Time
Solar-Mixer	**0.1482**	**0.1970**	**0.9586**	79	**0.1355**	**0.1868**	**0.9620**	64	**0.1239**	**0.1770**	**0.9652**	76	**0.1224**	**0.1762**	**0.9656**	76
LSTM	0.2167	0.2611	0.9401	239	0.2237	0.2789	0.9382	368	0.2794	0.3149	0.9232	1655	0.3081	0.3237	0.9153	1658
Seq-to-Seq	0.2712	0.3012	0.9252	840	0.2258	0.2490	0.9377	1811	0.2292	0.2618	0.9369	2727	0.2341	0.2915	0.9354	4050
MLP	0.1550	0.2127	0.9568	35	0.1461	0.2014	0.9591	47	0.2311	0.2928	0.9367	63	0.2076	0.2808	0.9428	58
LightGBM	0.4877	0.3537	0.8613	303	0.4624	0.4968	0.8717	696	0.4115	0.4766	0.8865	1189	0.3985	0.4737	0.8902	1528
Informer*	0.5571	0.6087	0.4168	866	1.1448	0.9074	-0.2012	458	1.0444	0.8567	-0.1041	537	1.1967	0.9070	-0.2584	568
FEDformer*	0.5205	0.4378	0.4551	666	0.5037	0.4198	0.4715	1349	0.4789	0.4279	0.4937	1368	0.4678	0.4183	0.5081	1281
Input Length	480				576				672				768			
Metrics	MSE	MAE	R2	Time	MSE	MAE	R2	Time	MSE	MAE	R2	Time	MSE	MAE	R2	Time
Solar-Mixer	**0.1239**	**0.1820**	**0.9652**	86	**0.1298**	**0.1858**	**0.9637**	89	**0.1207**	**0.1756**	**0.9660**	124	**0.1244**	**0.1815**	**0.9650**	95
LSTM	0.2214	0.2676	0.9391	1390	0.2229	0.2719	0.9386	1236	0.2099	0.2665	0.9422	1369	0.2409	0.2985	0.9342	1555
Seq-to-Seq	0.2610	0.3242	0.9278	4860	0.1758	0.2460	0.9515	5950	0.1643	0.2219	0.9545	7450	0.1714	0.2465	0.9524	9841
MLP	0.1945	0.2645	0.9464	62	0.1536	0.2212	0.9572	72	0.1923	0.2786	0.9471	76	0.1906	0.2783	0.9474	80
LightGBM	0.4080	0.4744	0.8875	1724	0.5330	0.5161	0.8535	2091	0.8248	0.6003	0.7774	2539	1.0560	0.6670	0.7146	2758
Informer*	1.0880	0.8970	-0.1521	667	1.1902	0.9288	-0.2553	620	1.0441	0.8467	-0.1071	403	1.0459	0.8883	-0.1134	422
FEDformer*	0.4409	0.4144	0.5331	1317	0.4181	0.4141	0.5590	1486	0.3616	0.3836	0.6166	1390	0.4336	0.4391	0.5384	1817

*: These models with early stopping methods, may have fewer training epochs, and the actual computational overhead should be much larger. Note that due to cuda out of memory error, the batch size of the lstm model is changed to 128 after the input length is greater than 192.

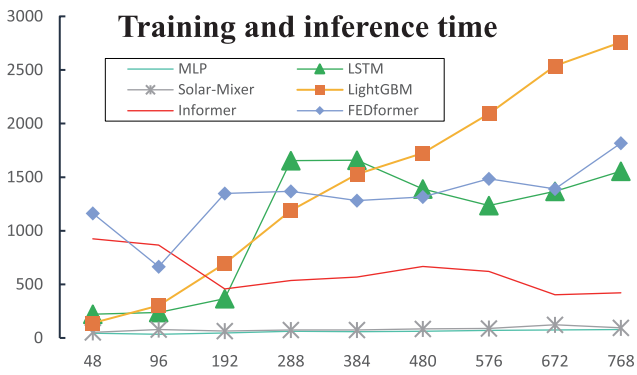


Fig. 7. Comparison of training and inference time cost for each model for different input lengths. Note that because of the excessive time consumption of the Seq-to-Seq model, it is not shown here for the sake of clarity of comparison of other models.

E. Influence of Input Length on the Model

In site 37 interval 2 data, an experiment was performed to observe the impact of different input lengths on the model for multivariate time series forecasting (accuracy and training cost). Specifically, Fig. 6 and Table VII shows the variation of the three metrics for each model with different input lengths. As the input length increases, all three metrics of the proposed model show some positive changes, while the other models show little change or negative changes.

On the other hand, the impact of different input lengths on model training and inference costs is presented in Fig. 7 and Table VII. Experimental results show that our model has a nice property that the training and inference costs grow slowly as the input length grows. This property is consistent with the MLP and Informer models. As a comparison, LightGBM is simple and easy to use, but its training cost increases sharply as the input length increases.

Combining the experimental results in this section, our model not only achieves better results with a fixed input length but also can receive longer inputs and capture more information in longer inputs to obtain better results without significantly increasing the training and inference overheads compared to other models.

VII. CONCLUSION

High-quality PPF methods are important for PV system planning and power scheduling. Short-term PPF does solve some problems in practical applications, but in some scenarios, it may require much longer time for power scheduling and operation. Therefore, accurate and efficient long-sequence PPF is essential. In this paper, a novel multivariate long-sequence PPF model called **Solar-Mixer** is proposed to improve the long-sequence PPF precision. **Solar-Mixer** consists of an anomaly detection and correction module and a forecasting module. The anomaly detection and correction module are designed to ensure less loss of valid information. The forecasting module combines a decomposition module and an MLP-Mixer feature extraction method, which is improved for time series forecasting tasks.

To verify the validity of **Solar-Mixer**, the proposed model was subjected to ablation studies, comparison experiments, and input length sensitivity experiments. Among them, the comparison experiments compare the effects of multiple models over a total of 4 time intervals at 3 sites. The compared models include MLP, LSTM, Seq-to-Seq, LightGBM, Informer, FEDformer, the state-of-the-art long-sequence time series forecasting model based on self-attention mechanisms. From these experiments, the following conclusions can be drawn:

1) Based on the results of the ablation study, our design for the model is effective in improving the forecasting ability of the model for time series.

Authorized licensed use limited to: Mukesh Patel School of Technology & Engineering. Downloaded on July 21,2026 at 04:29:54 UTC from IEEE Xplore. Restrictions apply.

2) By comparing **Solar-Mixer** with other models, it can be seen that **Solar-Mixer** has the best forecasting results for the same inputs.
3) The proposed model not only has advantages in multivariate time series forecasting but also can beat other models in most scenarios among univariate time series forecasting.
4) On the other hand, it is found that **Solar-Mixer** can receive longer input series without a significant increase in computational overhead. And increasing the length of the input series also brings an improvement in prediction accuracy.
5) The training time of the proposed model is only about 5% of that of the state-of-the-art time series forecasting model, and the proposed model still achieves the best prediction results.

REFERENCES

[1] C. Croonenbroeck and G. Stadtmann, "Renewable generation forecast studies–review and good practice guidance," *Renewable Sustain. Energy Rev.*, vol. 108, pp. 312–322, 2019.
[2] T. Qiu et al., "Potential assessment of photovoltaic power generation in China," *Renewable Sustain. Energy Rev.*, vol. 154, 2022, Art. no. 111900.
[3] T. Yao et al., "Intra-hour photovoltaic generation forecasting based on multi-source data and deep learning methods," *IEEE Trans. Sustain. Energy*, vol. 13, no. 1, pp. 607–618, Jan. 2022.
[4] J. Simeunović, B. Schubnel, P.-J. Alet, and R. E. Carrillo, "Spatio-temporal graph neural networks for multi-site PV power forecasting," *IEEE Trans. Sustain. Energy*, vol. 13, no. 2, pp. 1210–1220, Apr. 2022.
[5] H. Zhou et al., "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, 2021, vol. 35, pp. 11106–11115.
[6] M. Gao, J. Li, F. Hong, and D. Long, "Day-ahead power forecasting in a large-scale photovoltaic plant based on weather classification using lstm," *Energy*, vol. 187, 2019, Art. no. 115838.
[7] H. Zhang, Z. Lu, W. Hu, Y. Wang, L. Dong, and J. Zhang, "Coordinated optimal operation of hydro–wind–solar integrated systems," *Appl. Energy*, vol. 242, pp. 883–896, 2019.
[8] F. Angizeh and M. Parvania, "Stochastic scheduling of onsite solar power generation for large customers," in *Proc. IEEE Int. Conf. Probabilistic Methods Appl. to Power Syst.*, 2018, pp. 1–6.
[9] T. Dimri, S. Ahmad, and M. Sharif, "Time series analysis of climate variables using seasonal arima approach," *J. Earth Syst. Sci.*, vol. 129, no. 1, pp. 1–16, 2020.
[10] S. Ding, R. Li, and Z. Tao, "A novel adaptive discrete grey model with time-varying parameters for long-term photovoltaic power generation forecasting," *Energy Convers. Manage.*, vol. 227, 2021, Art. no. 113644.
[11] W. Sharika, L. Fernando, A. Kanagasundaram, R. Valluvan, and A. Kaneswaran, "Long-term solar irradiance forecasting approaches-a comparative study," in *Proc. IEEE Int. Conf. Inf. Automat. Sustainability*, 2018, pp. 1–6.
[12] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 3149–3157.
[13] T. Chen et al., "Xgboost: Extreme gradient boosting," *R Package Version 0.4-2*, vol. 1, no. 4, pp. 1–4, 2015.
[14] H. Li, Z. Ren, Y. Xu, W. Li, and B. Hu, "A multi-data driven hybrid learning method for weekly photovoltaic power scenario forecast," *IEEE Trans. Sustain. Energy*, vol. 13, no. 1, pp. 91–100, Jan. 2022.
[15] A. C. Ozdemir, K. Buluş, and K. Zor, "Medium-to long-term nickel price forecasting using LSTM and GRU networks," *Resour. Policy*, vol. 78, 2022, Art. no. 102906.
[16] K. K. R. Samal, A. K. Panda, K. S. Babu, and S. K. Das, "Multi-output TCN autoencoder for long-term pollution forecasting for multiple sites," *Urban Climate*, vol. 39, 2021, Art. no. 100943.
[17] T. Bogaerts, A. D. Masegosa, J. S. Angarita-Zapata, E. Onieva, and P. Hellinckx, "A graph CNN-LSTM neural network for short and long-term traffic forecasting based on trajectory data," *Transp. Res. Part C: Emerg. Technol.*, vol. 112, pp. 62–77, 2020.
[18] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition transformers with auto-correlation for long-term series forecasting," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 22419–22430.
[19] T. Zhou, Z. Ma, Q. Wen, X. Wang, L. Sun, and R. Jin, "FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *Proc. 39th Int. Conf. Mach. Learn.*, Jul. 2022, vol. 162, pp. 27268–27286. [Online]. Available: https://proceedings.mlr.press/v162/zhou22g.html
[20] S. Liu et al., "Pyraformer: Low-complexity pyramidal attention for long-range time series modeling and forecasting," in *Proc. Int. Conf. Learn. Representations*, 2022.
[21] A. Vaswani et al., "Attention is all you need," in *Proc. 31st Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.
[22] [Online]. Available: http://dkasolarcentre.com.au/source/alice-springs
[23] J. Zhou et al., "SDWPF: A dataset for spatial dynamic wind power forecasting challenge at KDD cup 2022," 2022, *arXiv:2208.04360*.
[24] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proc. IEEE 8th Int. Conf. Data Mining*, 2008, pp. 413–422.
[25] I. O. Tolstikhin et al., "MLP-mixer: An all-MLP architecture for vision," in *Proc. Adv. Neural Inf. Process. Syst.*, 2021, vol. 34, pp. 24261–24272.
[26] M. Li, X. Zhao, C. Lyu, M. Zhao, R. Wu, and R. Guo, "MLP4Rec: A pure MLP architecture for sequential recommendations," in *Proc. 31st Int. Joint Conf. Artif. Intell.*,, 2022, pp. 2138–2144. [Online]. Available: https://doi.org/10.24963/ijcai.2022/297
[27] F. Fusco, D. Pascual, and P. Staar, "pNLP-mixer: An efficient all-MLP architecture for language," 2022, *arXiv:2202.04350*.
[28] J. L. Ba, J. R. Kiros, and G. E. Hinton, "Layer normalization," 2016, *arXiv:1607.06450*.
[29] H. Touvron et al., "Resmlp: Feedforward networks for image classification with data-efficient training," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 45, no. 4, pp. 5314–5321, Apr. 2023.



Ziyuan Zhang received the B.E. degree from Xidian University, Xi'an, China, in 2019. He is currently working toward the master's degree with the Macau University of Science and Technology, Macao, China. His research interests include neural network-based time series prediction, deep learning, and renewable energy prediction.



Jianzhou Wang received the B.S. degree from Northwest Normal University, Lanzhou, China, in 1988, and the M.Sc. and Ph.D. degrees from Lanzhou University, Lanzhou, China, in 1998 and 2004, respectively. He is currently a Professor with the Faculty of Innovation Engineering, Macau University of Science and Technology, Macao, China. He has authored or coauthored more than 230 refereed journal papers. His research interests include wind energy forecasting, data mining, machine learning, statistical learning, and forecast theory.



Yurui Xia received the B.E. degree in automation in 2021 from Guangxi University. He is currently working toward a master's degree at the Macau University of Science and Technology, China. His research interests include evolutionary computation, complex networks, machine learning, and deep learning.

Authorized licensed use limited to: Mukesh Patel School of Technology & Engineering. Downloaded on July 21,2026 at 04:29:54 UTC from IEEE Xplore. Restrictions apply.



Danxiang Wei received the B.Ec. degree from the Dongbei University of Finance and Economics, Dalian, China, in 2020. He is currently working toward the Master of intelligent technology degree with the Macau University of Science and Technology, Macao, China. He has authored or coauthored several refereed journal papers on forecasting, statistical modeling, and energy management and engineering. His research interests include computational intelligence, deep learning, machine learning, and statistical modeling.



Yunbo Niu received the bachelor's degree from the Heilongjiang University of Science and Technology, Harbin, China, in 2021. He is currently working toward the master's degree with the Macau University of Science and Technology, Macao, China. His main research interests include neural networks, deep learning, machine learning, and time series prediction.

Authorized licensed use limited to: Mukesh Patel School of Technology & Engineering. Downloaded on July 21,2026 at 04:29:54 UTC from IEEE Xplore. Restrictions apply.